

Verilog In One Day

<https://www.asic-world.com/verilog/veritut.html>

Basic Verilog Syntax

- Module definition syntax

```
module priority_decoder_4to2 (  
    input wire [3:0] in,          // 4 input request lines  
    output reg [1:0] code,        // encoded index of highest-priority input  
    output reg          valid     // 1 if any input is asserted  
> ); ...  
  
endmodule
```

- `input/output` are keywords to define direction of port (`inout` port can be used for both)
- `// ...` and `/* */` are for comments
- `wire/reg` define data types and `[N:0]` defines size; we will get to them later
- Each module definition must define ports (inputs & outputs) in parentheses
- Module definition ends with `endmodule`
- **Note:** Curly braces are replaced with explicit `begin, end...`-style statements in Verilog

Calling a Module

- define variables for **input/output**
for eg: `req_lines`, `priority_code`,
`req_valid`
- instantiate it with a **unique instance name**
for eg: `pdec_inst` is the unique identifier for this
instance of `priority_decoder_4to2`
- wire in the inputs/outputs using **port names**
for eg: `.in(req_lines)` wires in `req_lines` to the `in` port

```
reg [3:0] req_lines;    // input - reg
wire [1:0] priority_code; // output - wire
wire      req_valid;    // output - wire

priority_decoder_4to2 pdec_inst (
    .in      (req_lines),
    .code    (priority_code),
    .valid    (req_valid)
);
```

Data Types

Verilog supports two logical data types:

- **wire** – carries a value **continuously** from whatever drives it; has no memory of its own
 - **assign** is used for **continuous assignment** to a wire. It cannot be used for register data.
 - Eg: `assign q_bar = ~q_reg;`
- **reg** – a variable that **holds its value** until explicitly assigned inside a procedural block
 - **Note:** The definition is **logical**. It is not guaranteed to synthesize to a **physical register**
 - Can only be assigned inside of `always @(...)` blocks

Two ways to do assignment

- Inside **always** blocks we can write **combinational** or **sequential** logic

Combinational

- requires **no physical registers**
- **ALL updates are executed at once**
- (typically) blocking assignment (**=**) is used

```
always @(*) begin
    c = a & b;
    d = a | b;
    out1 = d;
    out2 = c ^ b;
end
```

Sequential

- (usually) requires **physical registers**
- updates transferred **sequentially ONE-by-ONE**
- (typically) non-blocking assignment (**<=**) is used

```
always @(posedge clk) begin
    if (!rst_n)
        q_reg <= 1'b0;
    else
        q_reg <= d;
end
```

Back to Basics

Most C-style **operators** are supported:

- Bitwise: `&` `|` `^` `~` — per-bit operations
- Logical: `&&` `||` `!` — collapse to true/false, used in conditions
- Comparison: `==` `!=` `<` `>`
- Concatenation: `{a, b}` — combines signals into a wider signal vector

Control constructs (allowed inside `always` / `initial` only)

- `if` / `else` — same as software
- `case` — switch-statement for hardware

More data types

Verilog provides helper data types to make life easier – these do not map to real hardware (**wire/reg** do).

- **integer** – plain whole number variable, mostly used to count things (like loop iterations)

for eg: `integer i; for (i = 0; i < 16; i = i + 1) .`

- **parameter** – a named constant you can **reconfigure** from outside the module

for eg: `parameter WIDTH = 8` – can write variable-width modules using this

One isⁿt gud genug

Using vector signals in verilog is easy:

- **Declaring:** `wire/reg [MSB:LSB] var_name`

for eg: `wire [3:0] data` declares a wire that can carry 4 bits

- **Literals:** `<width>'<base><value>`

for eg: `4'b1010` (binary), `8'hA5` (hex), `4'd9` (decimal)

- **Assignment:**

`data = 4'b0110` (full assignment)

`data[1:0] = 2'b10` (multi-bit assignment)

`data[3:0] = {2'b01, 2'b10}` (using concat operator)

> this will assign the bits 3, 2, 1, 0 (MSB on the left to LSB on the right) to the values 0, 1, 1, 0 resp.

- **Reading:** Same syntax as assignment

for eg: `data[high:low]` will read bits `low, low+1, ... high-1, high`

- Note: You can use the `[LSB:MSB]` convention as well in Verilog – stay consistent with one convention.

Testing Verilog

```
`timescale 1ns/1ns

module tb;
    reg a, b, cin;
    wire cout, sum;

    adder uut (
        .a(a),
        .b(b),
        .cin(cin),
        .cout(cout),
        .sum(sum)
    );

    initial begin
        $monitor("At time %0t: a = %b, b = %b, \
        cin = %b -> cout = %b, sum = %b",
        | | | $time, a, b, cin, cout, sum);
    end

    initial begin
        $dumpfile("task1.vcd");
        $dumpvars(0, tb);
    end
endmodule
```

```
initial begin
    $dumpfile("task1.vcd");
    $dumpvars(0, tb);

    a = 0; b = 0; cin = 0; #10;
    a = 0; b = 0; cin = 1; #10;
    a = 0; b = 1; cin = 0; #10;
    a = 0; b = 1; cin = 1; #10;
    a = 1; b = 0; cin = 0; #10;
    a = 1; b = 0; cin = 1; #10;
    a = 1; b = 1; cin = 0; #10;
    a = 1; b = 1; cin = 1; #10;

    $display("Done!")
    $finish;
end
endmodule
```

Testing Verilog

```
`timescale 1ns/1ns

module tb;
    reg a, b, cin;
    wire cout, sum;

    adder uut (
        .a(a),
        .b(b),
        .cin(cin),
        .cout(cout),
        .sum(sum)
    );

    initial begin
        $monitor("At time %0t: a = %b, b = %b, \
        cin = %b -> cout = %b, sum = %b",
        |      |      | $time, a, b, cin, cout, sum);
    end

    initial begin
        $dumpfile("task1.vcd");
        $dumpvars(0, tb);
    end
endmodule
```

> A testbench for task1

- ``timescale <unit>/<precision>` for simulation
- N time units are represented as `#N`.
- use **reg** to store **inputs**, and **wire** to take **outputs**
- **instantiate** the module as **uut** and drive inputs/outputs for testing
- **initial** block will **run once** at T=0; used for setup code
- **\$monitor** prints a message anytime **one of the inputs changes**

Testing Verilog

```
initial begin
    $dumpfile("task1.vcd");
    $dumpvars(0, tb);

    a = 0; b = 0; cin = 0; #10;
    a = 0; b = 0; cin = 1; #10;
    a = 0; b = 1; cin = 0; #10;
    a = 0; b = 1; cin = 1; #10;
    a = 1; b = 0; cin = 0; #10;
    a = 1; b = 0; cin = 1; #10;
    a = 1; b = 1; cin = 0; #10;
    a = 1; b = 1; cin = 1; #10;

    $display("Done!");
    $finish;
end
endmodule
```

- **\$dumpfile** and **\$dumpvars** tell the simulator where to store waveform data and which variables to dump
- define delays in terms of the unit (**1ns here**) using **#10** etc.
- **\$display** prints a message once
- **\$finish** tells the simulator to end simulating

Testing Verilog

Output from `vvp` command:

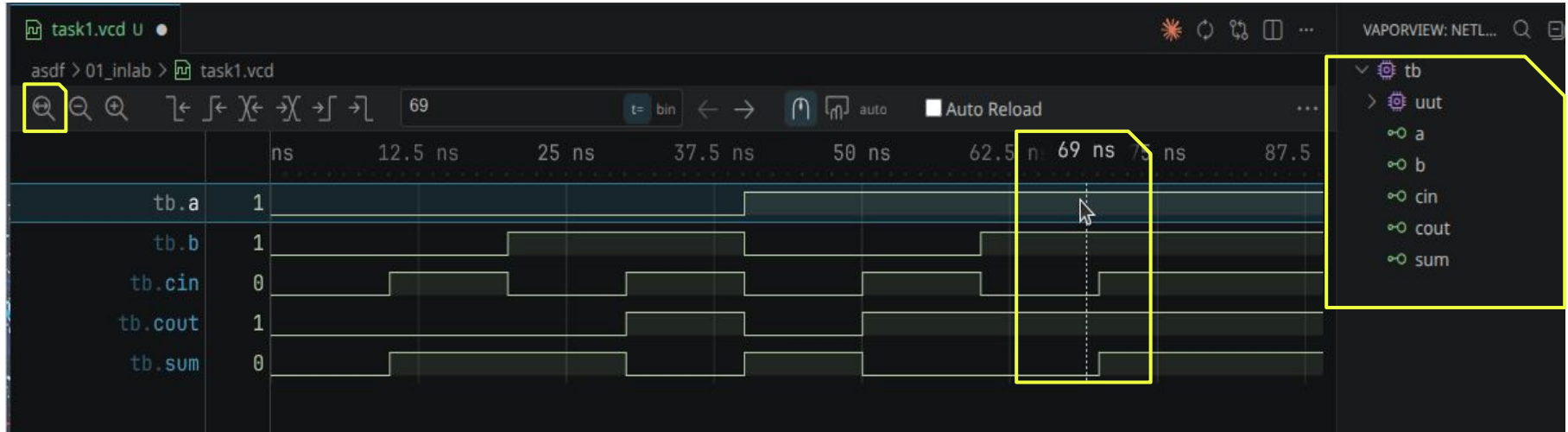
```
VCD info: dumpfile task1.vcd opened for output.  
At time 0: a = 0, b = 0,      cin = 0 -> cout = 0, sum = 0  
At time 10: a = 0, b = 0,     cin = 1 -> cout = 0, sum = 1  
At time 20: a = 0, b = 1,     cin = 0 -> cout = 0, sum = 1  
At time 30: a = 0, b = 1,     cin = 1 -> cout = 1, sum = 0  
At time 40: a = 1, b = 0,     cin = 0 -> cout = 0, sum = 1  
At time 50: a = 1, b = 0,     cin = 1 -> cout = 1, sum = 0  
At time 60: a = 1, b = 1,     cin = 0 -> cout = 1, sum = 0  
At time 70: a = 1, b = 1,     cin = 1 -> cout = 1, sum = 1  
Done!  
task1/tb_task1.v:35: $finish called at 80 (1ns)
```

Compiling Verilog

```
$ iverilog adder.v tb_task1.v -o task1.vvp  
$ vvp task1.vvp
```

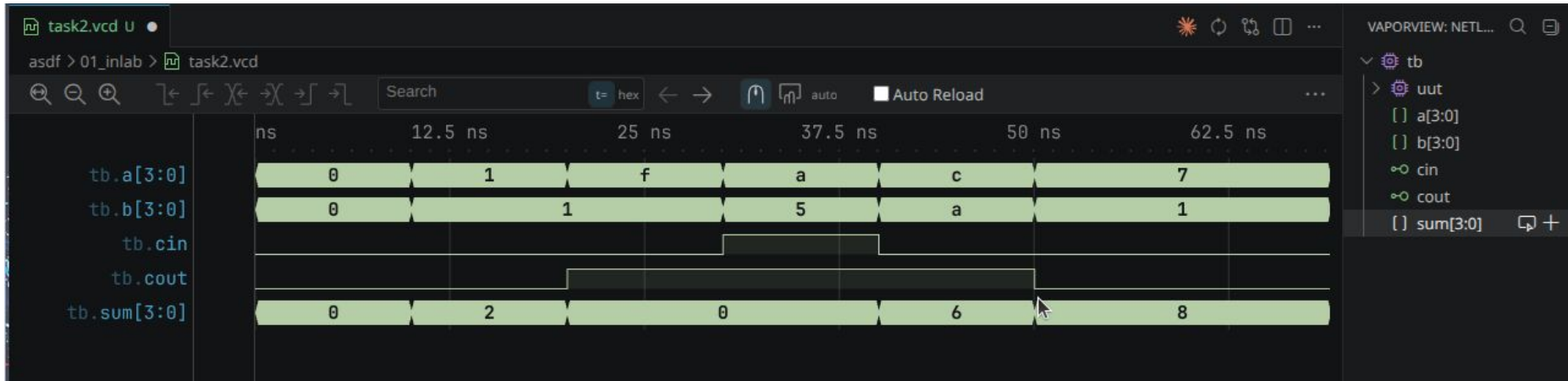
- First, compile **all verilog files** using **iverilog** (Icarus Verilog) into a **.vvp** output
- Second, **run the simulation using vvp**; this will execute the blocks in the **testbench** and display+dump suitable output
- Third, open the (optionally) generated **.vcd** file in **Vaporview** in **VS Code**

Seeing Verilog



- `$dumpfile/$dumpvars` are used to dump signal values over some simulation time to `.vcd` file
- you can view the waveforms evolving over time in a viewer such as GTKWave, or **Vaporview on VS Code**
- select the signals for display in the **Netlist View**, in the **Vaporview tab in the sidebar**
- click **Zoom to Fit**; then click anywhere (eg. 69ns here) to see values at that point

Seeing Verilog



- vector signals can also be inspected

It's inlab time

